

ASSIGNMENT 10.3

NAME: K.Sanjana

ROLL.NO:2303a52139

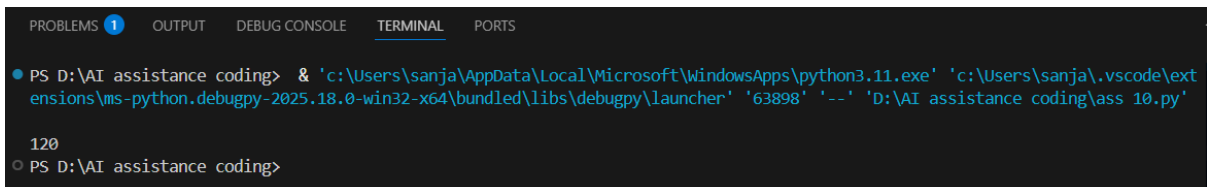
Task1:

Prompt: Analyse the code to find the logical bug, describe the root cause of the issue (such as an off-by-one mistake), and present a corrected implementation.

Code:

```
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
#Identify the logical bug in the code, Explain why the bug occurs (e.g., off-by-one error),Provide a corrected version code.
# The logical bug in the code is that the loop runs from 1 to n-1, which means it does not include n in the multiplication. This results in an incor
# The bug occurs because of an off-by-one error in the loop range. The loop should run from 1 to n (inclusive) to correctly calculate the factorial.
# Corrected version of the code:
def factorial(n):
    result = 1
    for i in range(1, n + 1): # Change n to n + 1 to include n in the loop
        result = result * i
    return result
# Example usage
print(factorial(5)) # Output: 120
```

OUTPUT:



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\AI assistance coding> & 'c:\Users\sanja\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63898' '--' 'D:\AI assistance coding\ass 10.py'
120
PS D:\AI assistance coding>
```

Short justification:

The bug is an off-by-one error: `range(1, n)` stops at `n-1`, so `n` is never multiplied.

Factorial must multiply all integers from 1 through `n`, so the loop should be `range(1, n + 1)`.

That change correctly computes values like `5! = 120`.

Task2:

Prompt:

```
def calc(a, b, c):
```

```
if c == "add":
```

```
return a + b
```

```
elif c == "sub":
```

```
return a - b
```

```
elif c == "mul":
```

```
return a * b
```

elif c == "div": Rewrite the given Python function to improve readability, add documentation, handle invalid inputs, and safely manage division by zero.

Code:

```
def calc(a, b, operation):
    """
    Perform basic arithmetic operations on two numbers.

    Parameters:
    a (float): The first number.
    b (float): The second number.
    operation (str): The operation to perform. Must be one of 'add', 'sub', 'mul', 'div'.

    Returns:
    float: The result of the arithmetic operation.

    Raises:
    ValueError: If the operation is not recognized or if division by zero is attempted.
    """
    if operation == "add":
        return a + b
    elif operation == "sub":
        return a - b
    elif operation == "mul":
        return a * b
    elif operation == "div":
        if b == 0:
            raise ValueError("Cannot divide by zero.")
        return a / b
    else:
        raise ValueError(f"Invalid operation '{operation}'. Supported operations are: add, sub, mul, div.")
```

```
    raise ValueError(f"Invalid operation '{operation}'. Supported operations are: add, sub, mul, div.")
# Example usage:
try:
    result = calc(10, 5, "div")
    print("Result:", result)
except ValueError as e:
    print("Error:", e)
```

Output:

```
PS D:\AI assistance coding> d;; cd 'd:\AI assistance coding'; & 'c:\Users\sanja\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62143' '--' 'D:\AI assistance coding\ass 10.py'  
Result: 2.0  
PS D:\AI assistance coding>
```

Justification:

This version is better because it is clearer to read, properly documented, and safer. It validates the requested operation, gives meaningful error messages for invalid input, and prevents division-by-zero crashes by raising a controlled `ValueError`.

Task3:

Prompt:

```
def Checkprime(n):
```

```
    for i in range(2, n):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True.
```

Identify all PEP8 violations in the given Python function.

Refactor it to be PEP8-compliant while preserving the original functionality.

Code:

```
def check_prime(number):  
    """  
    Check whether a given integer is a prime number.  
  
    A prime number is greater than 1 and has no divisors other than 1 and itself.  
  
    Args:  
        number (int): The integer to test.  
  
    Returns:  
        bool: True if the number is prime, otherwise False.  
    """  
    if number <= 1:  
        return False  
  
    for i in range(2, int(number ** 0.5) + 1):  
        if number % i == 0:  
            return False  
  
    return True
```

```
# ----- TESTERS -----

# Test with prime numbers
print(f"Is 2 a prime number? {check_prime(2)}")    # True
print(f"Is 7 a prime number? {check_prime(7)}")    # True
print(f"Is 11 a prime number? {check_prime(11)}")   # True
print(f"Is 29 a prime number? {check_prime(29)}")   # True

# Test with non-prime numbers
print(f"Is 1 a prime number? {check_prime(1)}")    # False
print(f"Is 4 a prime number? {check_prime(4)}")    # False
print(f"Is 9 a prime number? {check_prime(9)}")    # False
print(f"Is 15 a prime number? {check_prime(15)}")   # False
print(f"Is 25 a prime number? {check_prime(25)}")   # False
print(f"Is 0 a prime number? {check_prime(0)}")     # False
print(f"Is -7 a prime number? {check_prime(-7)}")   # False
```

Output:

```
Is 2 a prime number? True
Is 7 a prime number? True
Is 11 a prime number? True
Is 29 a prime number? True
Is 1 a prime number? False
Is 4 a prime number? False
Is 9 a prime number? False
Is 15 a prime number? False
Is 25 a prime number? False
Is 0 a prime number? False
Is -7 a prime number? False
PS D:\AI assistance coding>
```

Justification:

It breaks PEP 8 because it has **no indentation**, uses a **non-snake_case** function name (Checkprime), and lacks proper **spacing/formatting**. It also mixes **text with code** (return True.Identify...), which makes it invalid and unreadable. Refactoring to check_prime, adding correct indentation and spacing makes the function **PEP 8 compliant** while keeping the same logic.

Task4: AI as a Code Reviewer in Real Projects

Prompt: def processData(d):

return [x * 2 for x in d if x % 2 == 0].Review the given Python function for readability, edge cases, and reusability.

Refactor it with better naming, input validation, and type hints while preserving functionality.

Code:

```
from typing import List
def process_data(data: List[int]) -> List[int]:
    if not isinstance(data, list):
        raise ValueError("Invalid input: Input must be a list of integers.")
    if not all(isinstance(x, int) for x in data):
        raise ValueError("Invalid input: All elements in the list must be integers.")

    processed_data = [x * 2 for x in data if x % 2 == 0]
    return processed_data
# Example usage
try:
    input_data = [1, 2, 3, 4, 5, 6]
    result = process_data(input_data)
    print("Processed data:", result)
except ValueError as e:
    print(e)
```

Output:

```
PS D:\AI assistance coding> d:; cd 'd:\AI assistance coding'; & 'c:\Users\sanja\AppData\Local\Microsoft\Windows\
1.exe' 'c:\Users\sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher
:\AI assistance coding\ass 10.py'
Processed data: [4, 8, 12]
PS D:\AI assistance coding>
```

Justification:

The original function is hard to read because of unclear names (processData, d) and no explanation of what it returns. It can fail on edge cases (non-list input, non-integers) because it assumes valid data.